

Universidad
Manuela Beltrán
Acreditada en Alta Calidad Multicampus

GKMF – Garantía de Conocimiento y Movilidad Financiera

Documentación de análisis y diseño del sistema transaccional (Fase II)

Vallejo Bulla, Katherin Johana

Universidad Manuela Beltrán

Ingeniería de Software

Facultad de Ingeniería

Sistemas Transaccionales

Bogotá, Colombia

2026

Tabla de contenido

Introducción 6
1. Información general del proyecto 7
1.1 Nombre del sistema 7
1.2 Descripción general 7
1.3 Problemática o necesidad identificada 8
1.4 Justificación 8
1.5 Alcance de la solución 9
2. Objetivo central 10
3. Objetivos específicos 10
4. Arquitectura de software 11
4.1 Tipo de arquitectura seleccionado 11
4.2 Diagrama de la arquitectura 11
4.3 Componentes y funciones 12
4.4 Comunicación entre los componentes 13
4.5 Seguridad por diseño 14
4.6 Justificación de la arquitectura 15
4.7 Decisiones tecnológicas 16
5. Diagrama de casos de uso UML 17
5.1 Actores 17
5.2 Diagrama de casos de uso 17
5.3 Catálogo de casos de uso 20
5.4 Descripción de los principales casos de uso 20
6. Diagrama de clases UML 25
6.1 Criterios de modelado 25
6.2 Vista financiera y de seguridad 26
6.3 Vista educativa 28
6.4 Descripción de las clases 29
6.5 Relaciones y multiplicidades 30
6.6 Coherencia con los casos de uso 30
7. Modelo relacional 31
7.1 Criterios de diseño 31
7.2 Diagrama del modelo relacional 32
7.3 Diccionario de datos 34
7.4 Relaciones y cardinalidades 39
7.5 Restricciones de integridad 40
7.6 Índices 42
7.7 Correspondencia entre clases y tablas 42
7.8 Normalización 43
8. Integración de los modelos 44
8.1 Cadena de coherencia 44
8.2 Ejemplo integrador: depositar en una meta 45

8.3 Verificación de la correspondencia 46
9. Conclusiones 47
10. Referencias 48
Anexo A. Estructura del repositorio 50

Lista de tablas

Tabla 1 Ficha del proyecto 7
Tabla 2 Alcance del sistema 9
Tabla 3 Objetivos específicos y funcionalidades asociadas 10
Tabla 4 Componentes de la arquitectura y su función 13
Tabla 5 Flujos de comunicación entre los componentes 14
Tabla 6 Amenazas, controles de la arquitectura y respaldo en la base de datos 15
Tabla 7 Tecnologías por capa y su relación con la Fase I 17
Tabla 8 Actores del sistema 17
Tabla 9 Catálogo de casos de uso 20
Tabla 10 Descripción del caso de uso CU-02 Iniciar sesión 21
Tabla 11 Descripción del caso de uso CU-05 Crear meta de ahorro 22
Tabla 12 Descripción del caso de uso CU-09 Depositar en meta de ahorro 23
Tabla 13 Descripción del caso de uso CU-10 Retirar de meta de ahorro 24
Tabla 14 Descripción del caso de uso CU-13 Realizar lección y registrar progreso 24
Tabla 15 Descripción del caso de uso CU-15 Sincronizar datos 25
Tabla 16 Clases del sistema, responsabilidades y tabla asociada 29
Tabla 17 Relaciones entre clases 30
Tabla 18 Correspondencia entre casos de uso y clases 30
Tabla 19 Criterios de diseño del modelo relacional 32
Tabla 20 Diccionario de datos de la tabla usuario 34
Tabla 21 Diccionario de datos de la tabla sesion 35
Tabla 22 Diccionario de datos de la tabla meta_ahorro 35
Tabla 23 Diccionario de datos de la tabla categoria_movimiento 36
Tabla 24 Diccionario de datos de la tabla transaccion 36
Tabla 25 Diccionario de datos de la tabla sincronizacion 37
Tabla 26 Diccionario de datos de la tabla categoria_leccion 37
Tabla 27 Diccionario de datos de la tabla leccion 38
Tabla 28 Diccionario de datos de la tabla progreso_leccion 38
Tabla 29 Diccionario de datos de la tabla logro 39
Tabla 30 Diccionario de datos de la tabla usuario_logro 39
Tabla 31 Claves foráneas, relaciones y cardinalidades 40
Tabla 32 Restricciones de unicidad (UQ) y de verificación (CK) 40
Tabla 33 Índices del modelo relacional 42
Tabla 34 Correspondencia entre el modelo de clases y el modelo relacional 43
Tabla 35 Trazabilidad desde la problemática hasta los casos de uso 44
Tabla 36 Trazabilidad desde las funcionalidades hasta la arquitectura 45
Tabla 37 Recorrido del caso de uso CU-09 Depositar en meta por todos los modelos 46
Tabla 38 Verificaciones realizadas sobre el modelo y su implementación 47
Tabla A1 Estructura del repositorio del proyecto 50

Lista de figuras

Figura 1 Arquitectura de software de GKMF 12
Figura 2 Diagrama de casos de uso del sistema GKMF 19
Figura 3 Diagrama de clases: vista financiera y de seguridad 26
Figura 4 Diagrama de clases: vista educativa 28
Figura 5 Modelo relacional de GKMF 33

Introducción

La inclusión financiera sigue siendo un reto para amplios sectores de la población colombiana que administran su dinero de manera informal y cuentan con pocas herramientas para aprender a manejarlo. GKMF (Garantía de Conocimiento y Movilidad Financiera) es una aplicación móvil que busca acompañar a esas personas en el ahorro por metas, el registro de sus movimientos y la educación financiera, incluso cuando la conexión a internet es limitada (Vallejo Bulla, 2026a).

En la Fase I se contextualizó la problemática y se construyó un prototipo de interfaz con cinco pantallas: carga, inicio, metas, aprendizaje y perfil (Vallejo Bulla, 2026b). Este documento corresponde a la Fase II y tiene por propósito transformar esa idea inicial en una propuesta técnica estructurada: define el objetivo central y los objetivos específicos, propone una arquitectura de software, modela el comportamiento del sistema mediante casos de uso, representa su estructura estática con un diagrama de clases y diseña el modelo relacional que soportará los datos.

El documento se organiza en diez secciones que siguen la cadena de coherencia problemática, objetivos, funcionalidades, casos de uso, clases, modelo relacional y arquitectura. La sección 8 demuestra esa coherencia mediante matrices de trazabilidad y un ejemplo que recorre todos los modelos. Los diagramas se basan en el estándar UML 2.5.1 (Object Management Group [OMG], 2017) y el modelo de datos, en el modelo relacional de Codd (1970).

Como evidencia de viabilidad, el modelo relacional se implementó y se verificó en el motor SQL Server, y se inició la estructura del backend; ambos artefactos se entregan en el repositorio del proyecto (Vallejo Bulla, 2026b). Las cifras del modelo que aparecen en este documento (11 tablas, 12 claves foráneas y 23 restricciones CHECK) provienen del catálogo de esa base de datos, por lo que documento e implementación coinciden.

1. Información general del proyecto

1.1 Nombre del sistema

El sistema se denomina **GKMF – Garantía de Conocimiento y Movilidad Financiera**. Es un sistema transaccional de gestión de ahorro personal y educación financiera con acceso desde dispositivos móviles. La Tabla 1 resume la ficha del proyecto.

Tabla 1
Ficha del proyecto

Campo	Descripción
Nombre del sistema	GKMF – Garantía de Conocimiento y Movilidad Financiera
Tipo de sistema	Aplicación móvil con backend transaccional (API REST) y base de datos relacional
Usuarios objetivo	Personas con acceso limitado al sistema financiero formal, estudiantes y jóvenes que inician su vida financiera
Modalidad de acceso	Aplicación móvil (Android e iOS) con modo sin conexión
Estudiante	Vallejo Bulla, Katherin Johana
Programa y nivel	Ingeniería de Software, sexto semestre
Asignatura	Sistemas Transaccionales
Institución	Universidad Manuela Beltrán, Bogotá, Colombia
Fase del documento	Fase II: análisis y diseño
Repositorio	https://github.com/Trabajos-Universidad-Manuela-Beltran/GKMF-Garant-a-de-Conocimiento-y-Movilidad-Financiera

Nota. Elaboración propia.

1.2 Descripción general

GKMF permite a una persona crear metas de ahorro, registrar los ingresos y gastos que declara, depositar y retirar dinero de sus metas, avanzar en lecciones cortas de educación financiera y obtener logros que refuerzan sus hábitos. La aplicación conserva los datos del usuario en el dispositivo para que pueda operar sin conexión y los sincroniza con un servidor central cuando la conectividad se restablece.

Desde el punto de vista técnico se trata de un sistema transaccional: cada operación que modifica dinero registrado (por ejemplo, un depósito en una meta) debe cumplir las propiedades

ACID —atomicidad, consistencia, aislamiento y durabilidad— para que el saldo que ve el usuario sea siempre correcto (Haerder y Reuter, 1983; Gray y Reuter, 1993).

1.3 Problemática o necesidad identificada

Como se documentó en la Fase I, en numerosas comunidades colombianas persisten dificultades para acceder a servicios financieros formales. Buena parte de la población de bajos ingresos maneja su dinero de manera informal (efectivo guardado en casa, cadenas de ahorro, prestamistas informales), lo que la expone a pérdidas, robos y condiciones desfavorables. A esto se suman el desconocimiento de conceptos financieros básicos y el hecho de que muchas soluciones de banca digital asumen conectividad permanente y procesos de vinculación exigentes (Vallejo Bulla, 2026a).

De esa problemática surge la necesidad de una herramienta accesible que permita gestionar el ahorro de forma segura y comprensible, que fortalezca la educación financiera del usuario y que funcione en condiciones reales de conectividad. Para este documento la problemática se descompone en cuatro necesidades específicas, que se usarán para verificar la coherencia entre los modelos: **P1**, manejo informal e inseguro del dinero; **P2**, bajo nivel de educación financiera; **P3**, conectividad limitada e intermitente; y **P4**, desconfianza en la integridad y la seguridad de las operaciones financieras digitales.

1.4 Justificación

El desarrollo de GKMF se justifica por su utilidad social y por su pertinencia académica. En lo social, ofrece un primer paso concreto hacia la autonomía económica de personas con baja bancarización, de estudiantes y de jóvenes, y puede servir de complemento a programas de educación financiera. En lo académico, exige aplicar en un caso real y sensible los conceptos

centrales de los sistemas transaccionales —integridad, consistencia, aislamiento y durabilidad— y las prácticas de diseño de arquitecturas seguras (Silberschatz et al., 2020).

Además, al manejar datos personales y financieros, el sistema debe respetar el régimen colombiano de protección de datos personales (Congreso de la República de Colombia, 2012). Esto orienta decisiones de diseño como minimizar los datos almacenados, guardar solo hashes de contraseñas y de tokens, y evitar el borrado físico de la información financiera.

1.5 Alcance de la solución

La Tabla 2 delimita lo que abarca el sistema y lo que queda fuera de él. GKMF registra y gestiona el ahorro que el usuario declara; en esta versión no custodia dinero ni procesa pagos, por lo que no es una entidad financiera ni sustituye a una.

Tabla 2
Alcance del sistema

Dentro del alcance	Fuera del alcance
Registro, autenticación y gestión de la cuenta y de la sesión del usuario	Custodia de fondos, pagos, transferencias o conexión con entidades bancarias
Metas de ahorro con seguimiento de progreso y estados	Apertura de productos financieros, créditos o inversiones reales
Registro de ingresos, egresos, depósitos y retiros de metas	Asesoría financiera personalizada y recomendaciones automáticas
Educación financiera: lecciones, progreso y logros, y su administración	Pasarelas de pago y facturación
Modo sin conexión y sincronización con el servidor	Cumplimiento de regulación de entidades vigiladas por la Superintendencia Financiera

Nota. Elaboración propia.

Pregunta problema. La Fase I formuló la siguiente pregunta, que orienta todo el diseño: «¿Cómo diseñar un sistema transaccional para la aplicación GKMF que garantice la integridad, consistencia y seguridad de las operaciones de ahorro de sus usuarios, incluso en condiciones de conectividad limitada, favoreciendo así su inclusión y educación financiera?» (Vallejo Bulla, 2026a).

2. Objetivo central

Diseñar el sistema transaccional de GKMF —su arquitectura de software, sus modelos UML y su modelo relacional— de modo que permita registrar y consultar metas de ahorro, movimientos financieros y avance educativo de forma íntegra, consistente y segura, incluso sin conexión permanente a internet, para favorecer la inclusión y la educación financiera de sus usuarios.

Este objetivo responde directamente a la pregunta problema de la sección 1.5: la integridad y la consistencia se abordan con el modelo relacional y las transacciones ACID; la seguridad, con los controles de la arquitectura; y la conectividad limitada, con el diseño *offline-first* y la sincronización.

3. Objetivos específicos

Para alcanzar el objetivo central se plantean cinco objetivos específicos, cada uno asociado a una funcionalidad principal del sistema y a la necesidad de la sección 1.3 que atiende (véase la Tabla 3).

Tabla 3
Objetivos específicos y funcionalidades asociadas

Código	Objetivo específico	Funcionalidad	Necesidad
OE1	Permitir el registro, la autenticación y la administración segura de usuarios y sesiones, con control de acceso por rol.	F1. Cuenta y acceso	P4
OE2	Gestionar metas de ahorro y registrar ingresos, egresos, depósitos y retiros como transacciones atómicas, consistentes y auditables.	F2. Metas de ahorro F3. Movimientos financieros	P1, P4
OE3	Ofrecer educación financiera mediante lecciones, seguimiento del progreso y logros que motiven al usuario.	F4. Educación financiera	P2
OE4	Habilitar el uso sin conexión y la sincronización idempotente entre el dispositivo y el servidor.	F5. Modo sin conexión y sincronización	P3
OE5	Establecer una arquitectura por capas y un modelo relacional que garanticen la integridad y la seguridad de la información.	F6. Integridad y seguridad de los datos (transversal)	P4

Nota. Elaboración propia.

4. Arquitectura de software

4.1 Tipo de arquitectura seleccionado

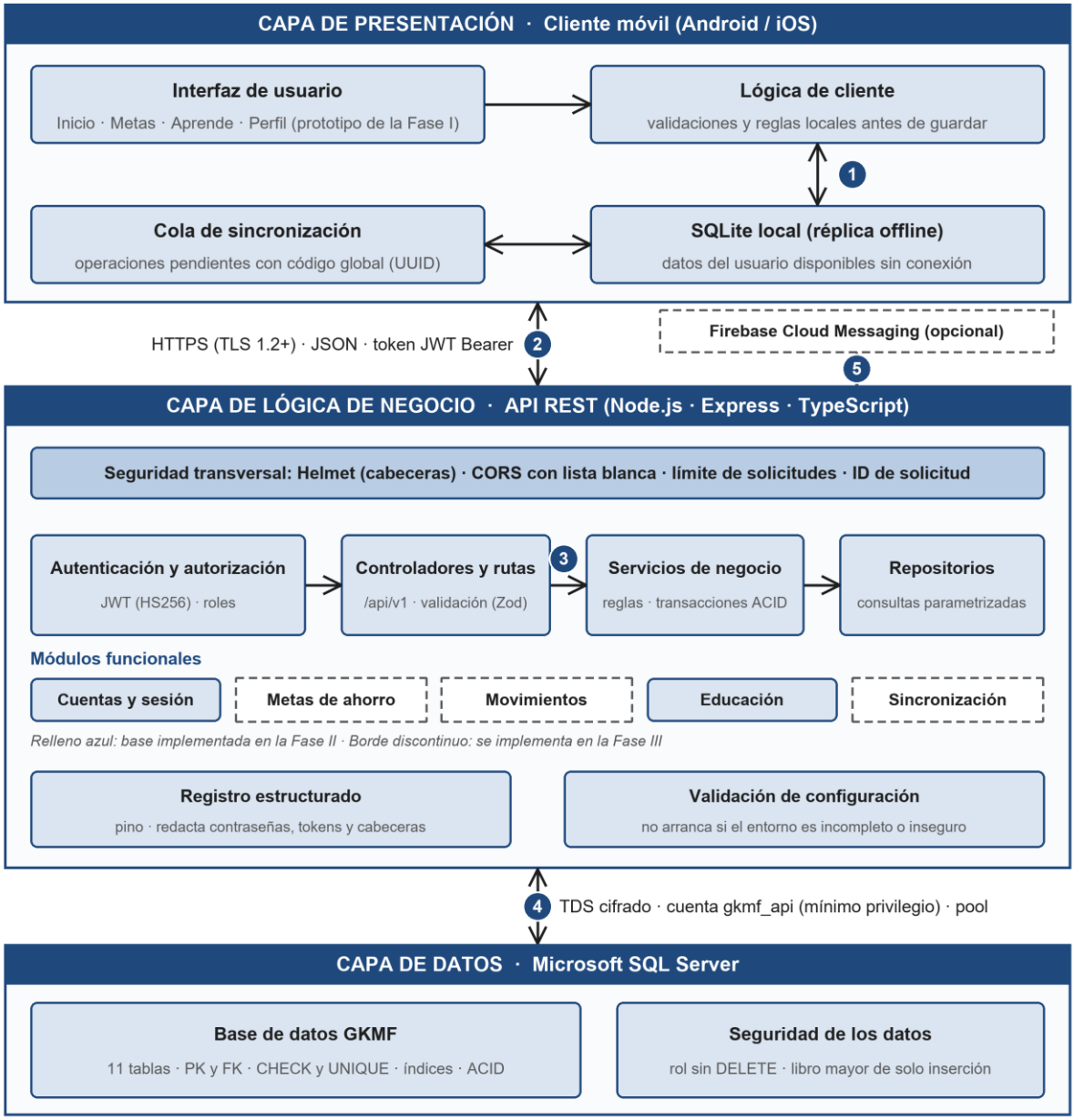
Se propone una **arquitectura en capas** con estilo **cliente–servidor**. El sistema se divide en tres capas lógicas —presentación, lógica de negocio y datos— y en tres nodos físicos: el dispositivo móvil del usuario, el servidor de aplicaciones que expone una API REST y el servidor de base de datos. Cada capa solo se comunica con la inmediatamente inferior, lo que reduce el acoplamiento y permite modificar o probar una capa sin afectar a las demás (Bass et al., 2021; Fowler, 2002).

Sobre esa base se agregan dos decisiones propias del problema: el cliente móvil sigue el enfoque *offline-first* (conserva una réplica local en SQLite y una cola de operaciones pendientes) y el servidor aplica el principio de defensa en profundidad, de modo que la seguridad no depende de un único componente.

4.2 Diagrama de la arquitectura

La Figura 1 representa la arquitectura. Los números 1 a 5 señalan los flujos de comunicación que se describen en la sección 4.4.

Figura 1
Arquitectura de software de GKMF



Nota. Elaboración propia. Las líneas discontinuas indican elementos externos u opcionales o funcionalidades de la Fase III.

4.3 Componentes y funciones

La Tabla 4 describe cada componente, la capa a la que pertenece y su función.

Tabla 4
Componentes de la arquitectura y su función

Componente	Capa	Función
Interfaz de usuario	Presentación	Muestra las pantallas Inicio, Metas, Aprende y Perfil y captura las acciones del usuario. Se basa en el prototipo de la Fase I.
Lógica de cliente	Presentación	Valida los datos antes de guardarlos (por ejemplo, que un monto sea positivo) y coordina el acceso a la base local y a la API.
SQLite local	Presentación (cliente)	Réplica local de los datos del usuario; permite consultar y registrar información sin conexión.
Cola de sincronización	Presentación (cliente)	Almacena las operaciones hechas sin conexión, cada una con un código global (UUID), hasta poder enviarlas.
Seguridad transversal	Lógica de negocio	Cabeceras seguras (Helmet), CORS con lista blanca, límite de solicitudes por IP, identificador de solicitud y límite del tamaño del cuerpo.
Autenticación y autorización	Lógica de negocio	Verifica el token JWT (firma, emisor, audiencia y expiración) y el rol del usuario antes de ejecutar una operación. Todo lo que no sea /salud exige token (denegación por defecto).
Controladores y rutas	Lógica de negocio	Exponen los recursos de la API en /api/v1, validan las entradas con esquemas Zod y devuelven respuestas con un formato uniforme.
Servicios de negocio	Lógica de negocio	Implementan las reglas (saldo nunca negativo, estados de la meta, logros) y orquestan las transacciones ACID.
Repositorios	Lógica de negocio	Acceden a la base de datos con consultas parametrizadas y un pool de conexiones; centralizan las transacciones.
Base de datos GKMF	Datos	Almacena las 11 tablas con sus claves, restricciones e índices; garantiza la integridad incluso ante errores del código.
Registro estructurado	Transversal	Registra cada solicitud y error sin exponer contraseñas, tokens ni cabeceras sensibles.
Firebase Cloud Messaging	Externo (opcional)	Envía notificaciones push de recordatorio, según lo previsto en la Fase I; no recibe datos financieros.

Nota. Elaboración propia.

4.4 Comunicación entre los componentes

La Tabla 5 describe cómo se comunican los componentes. El cliente nunca accede directamente a la base de datos central: toda operación pasa por la API, que es la única capa que conoce las credenciales de la base de datos.

Tabla 5
Flujos de comunicación entre los componentes

Flujo	Origen y destino	Medio y formato	Seguridad
1	Interfaz, lógica de cliente y SQLite local	Llamadas locales y consultas SQL locales	Los datos se guardan solo en el dispositivo del usuario.
2	Aplicación móvil y API	HTTPS (TLS 1.2 o superior), JSON y cabecera Authorization: Bearer	Token JWT de vida corta, validación de entradas y límite de solicitudes. Incluye el envío de los lotes de la cola de sincronización.
3	Rutas, servicios y repositorios	Llamadas entre funciones dentro del servidor	Cada capa solo llama a la inferior; los errores se traducen en un único manejador.
4	Repositorios y base de datos	Protocolo TDS cifrado, pool de conexiones y consultas parametrizadas	Cuenta gkmf_api de mínimo privilegio, sin permiso de borrado.
5	API y Firebase Cloud Messaging	HTTPS	Solo identificadores de notificación; sin datos financieros.

Nota. Elaboración propia.

La sincronización sigue un protocolo que evita duplicados. Primero, el dispositivo envía un lote con las operaciones pendientes. Segundo, el servidor registra una sincronización en estado EN_CURSO y aplica cada operación en su propia transacción. Tercero, si el código global de una operación ya existe, la descarta sin error (idempotencia), de modo que reenviar un lote no duplica movimientos. Finalmente, el servidor cierra la sincronización como COMPLETADA, PARCIAL o FALLIDA y devuelve el resultado por operación.

4.5 Seguridad por diseño

Por tratarse de información financiera y personal, la seguridad se incorporó desde el diseño y se contrastó con el OWASP API Security Top 10 (OWASP Foundation, 2023) y con el OWASP Top 10 (OWASP Foundation, 2021). La Tabla 6 relaciona cada amenaza con el control previsto en la arquitectura y con el elemento del modelo de datos que lo respalda.

Tabla 6

Amenazas, controles de la arquitectura y respaldo en la base de datos

Amenaza	Control en la arquitectura	Respaldo en la base de datos
Acceso a datos de otro usuario (API1:2023, autorización rota a nivel de objeto)	Toda consulta filtra por el usuario del token; el id del usuario nunca se toma del cuerpo de la solicitud.	Claves foráneas compuestas (id_meta, id_usuario) e (id_sincronizacion, id_usuario): un movimiento no puede referenciar la meta de otro usuario.
Autenticación débil (API2:2023)	Contraseñas con scrypt y sal única; tokens JWT de 15 minutos con algoritmo fijo; bloqueo temporal tras intentos fallidos; límite de solicitudes.	hash_contrasena con longitud mínima; intentos_fallidos y bloqueado_hasta; sesion guarda solo el hash del token de refresco.
Exposición excesiva de datos (API3:2023)	Los repositorios listan las columnas explícitamente (nunca SELECT *); las respuestas usan objetos de transferencia; los errores no revelan detalles internos.	El hash de la contraseña no sale de la capa de acceso a datos.
Consumo no controlado de recursos (API4:2023)	Límite de solicitudes por IP, cuerpo máximo de 100 kB y paginación acotada.	Índices sobre las consultas frecuentes.
Autorización a nivel de función (API5:2023)	Middleware autorizar(rol) en las operaciones de administración.	Restricción CK_usuario_rol; la cuenta de la API no puede modificar la columna rol.
Configuración insegura (API8:2023)	Cabeceras de Helmet, CORS con lista blanca, validación del entorno al arrancar, rechazo de configuraciones inseguras en producción y comprobación de que la cuenta de base de datos no es administrativa.	Conexión cifrada (TDS) y cuenta sin privilegios administrativos.
Inyección (A03:2021)	Consultas parametrizadas y validación estricta con Zod, que rechaza campos desconocidos.	Tipos de datos y restricciones CHECK como segunda barrera.
Manipulación del libro de movimientos	Los movimientos son de solo inserción; las correcciones se hacen con movimientos inversos.	La cuenta de la API no tiene UPDATE ni DELETE sobre transaccion.
Pérdida de trazabilidad (A09:2021)	Registro estructurado con identificador de solicitud y sin datos sensibles.	fecha_registro y codigo_global en cada movimiento; tabla sincronizacion.
Uso indebido de datos personales	Minimización de datos y baja lógica (activo = 0) en lugar de borrado físico.	Sin borrado en cascada: las claves foráneas no eliminan filas hijas.

Nota. Elaboración propia.

4.6 Justificación de la arquitectura

La arquitectura en capas es adecuada para GKMF por cuatro razones. Primero, separa responsabilidades: la interfaz cambia con frecuencia, las reglas financieras cambian poco y los datos deben preservarse, y aislarlas facilita el mantenimiento y las pruebas de cada parte. Segundo, centraliza la seguridad: al pasar todo por la API, los controles de autenticación, validación y límite de uso se aplican en un solo lugar. Tercero, encapsula las transacciones: el acceso a datos reside en los repositorios, donde una operación como depositar en una meta se

ejecuta dentro de una única transacción ACID. Cuarto, es una arquitectura conocida y bien documentada, lo que reduce el riesgo en un proyecto académico individual (Sommerville, 2016).

El estilo cliente–servidor con API REST permite que el cliente móvil sea liviano y que la lógica crítica resida en el servidor, donde es posible auditarla y protegerla. REST impone comunicaciones sin estado y una interfaz uniforme, lo que facilita el escalado y permite versionar la API sin romper a los clientes ya instalados (Fielding, 2000).

Finalmente, el enfoque *offline-first* responde a la necesidad P3: un usuario con conectividad intermitente puede seguir registrando movimientos y metas y sincronizarlos después. Como un mismo dispositivo puede reenviar un lote varias veces, cada operación lleva un identificador global (codigo_global) que el servidor usa para aplicarla una sola vez. Se eligió un modelo relacional con transacciones ACID, y no uno documental, porque los saldos, los movimientos y sus restricciones son datos fuertemente estructurados en los que la integridad referencial es un requisito y no una opción (Codd, 1970; Elmasri y Navathe, 2016).

4.7 Decisiones tecnológicas

La Tabla 7 resume las tecnologías elegidas. En la Fase I se había planteado Firebase Firestore como almacén principal y SQLite como base local. Al formalizar el modelo relacional se decidió usar Microsoft SQL Server como fuente única de verdad, porque una base documental como Firestore no soporta claves foráneas ni restricciones de integridad, que son el núcleo de este diseño. SQLite se conserva como réplica local en el dispositivo y Firebase Cloud Messaging se mantiene para las notificaciones.

Tabla 7
Tecnologías por capa y su relación con la Fase I

Capa	Tecnología	Justificación	Frente a la Fase I
Cliente móvil	React Native o Flutter (se define en la Fase III); prototipo HTML, CSS y JavaScript	Un solo código para Android e iOS.	Sin cambio
Base local	SQLite	Base relacional embebida, sin servidor, apta para el uso sin conexión.	Se mantiene
API	Node.js, Express y TypeScript	Tipado estático, ecosistema maduro de seguridad y el mismo lenguaje del prototipo.	Reemplaza a Firebase como backend
Validación y seguridad	Zod, Helmet, express-rate-limit, jose (JWT) y scrypt	Validación estricta y controles ya probados en producción.	Nuevo
Base de datos central	Microsoft SQL Server (T-SQL)	Transacciones ACID, claves foráneas, restricciones CHECK e índices filtrados; es la herramienta usada en la Guía 1 de la asignatura.	Reemplaza a Firestore
Notificaciones	Firebase Cloud Messaging	Ya previsto en la Fase I; no maneja datos financieros.	Se mantiene
Entorno	Docker (SQL Server y API)	Entorno reproducible para desarrollo y pruebas.	Nuevo

Nota. Elaboración propia.

5. Diagrama de casos de uso UML

5.1 Actores

En el sistema interactúan dos actores humanos, descritos en la Tabla 8. Una persona que aún no inició sesión solo puede ejecutar los casos de uso CU-01 y CU-02.

Tabla 8
Actores del sistema

Actor	Descripción	Casos de uso
Usuario	Persona registrada que gestiona su ahorro, registra movimientos y estudia educación financiera. Corresponde al rol USUARIO.	CU-01 a CU-15
Administrador de contenido	Persona con el rol ADMINISTRADOR que mantiene el catálogo de lecciones, categorías y logros.	CU-16

Nota. Elaboración propia.

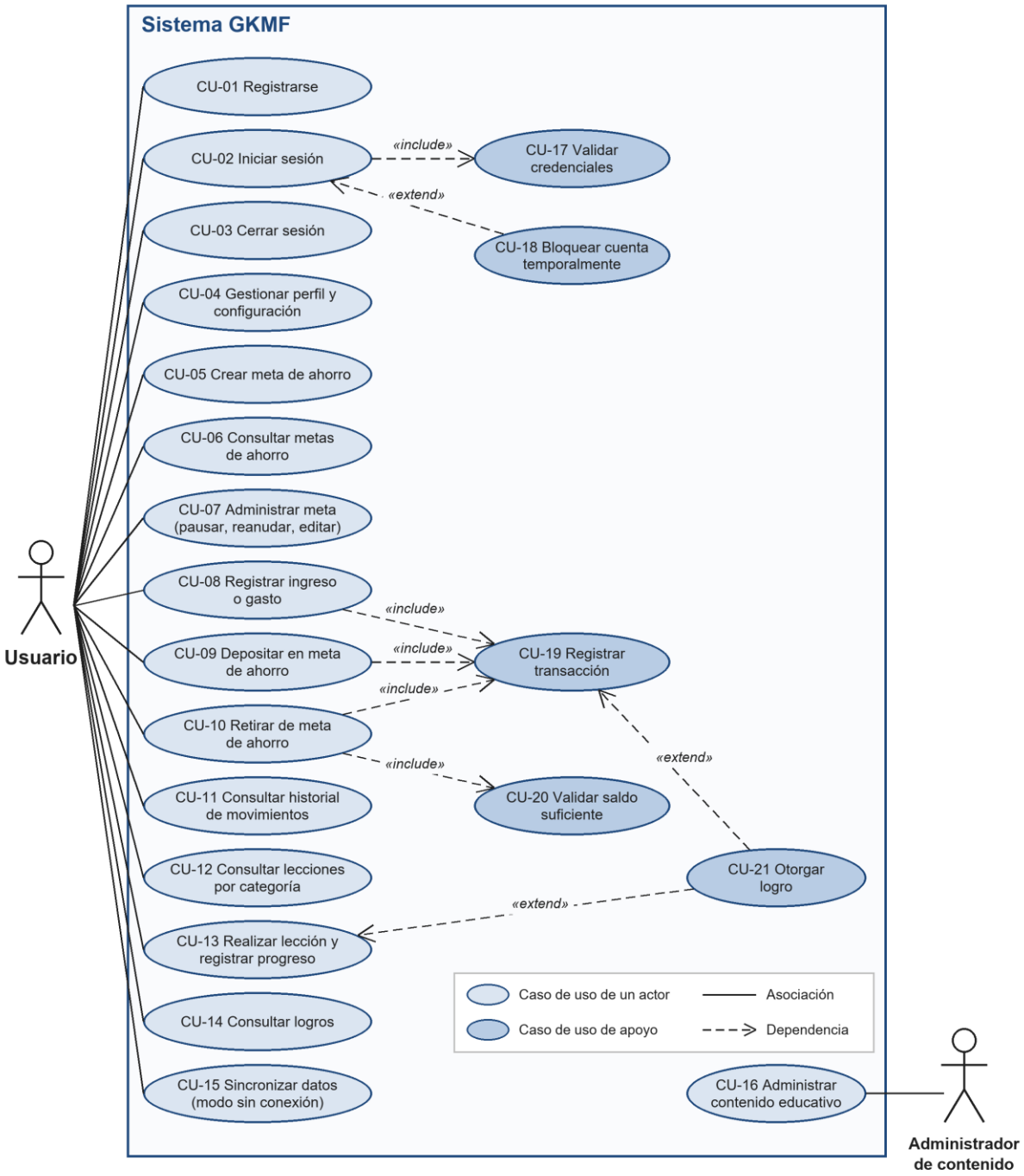
5.2 Diagrama de casos de uso

La Figura 2 presenta el diagrama de casos de uso. Los casos CU-01 a CU-16 son funcionalidades que percibe el actor; los casos CU-17 a CU-21, en azul más oscuro, son casos de

apoyo que se reutilizan mediante relaciones «include» o «extend». En una relación «include», el caso base siempre ejecuta al incluido; en una «extend», el caso de extensión se ejecuta solo si se cumple una condición (por ejemplo, un logro se otorga únicamente cuando se cumple su criterio).

Figura 2

Diagrama de casos de uso del sistema GKMF



Nota. Elaboración propia con notación UML 2.5.1 (Object Management Group [OMG], 2017).

5.3 Catálogo de casos de uso

La Tabla 9 lista los casos de uso con su actor, la funcionalidad del sistema a la que pertenecen (F1 a F6, según la Tabla 3) y la pantalla del prototipo de la Fase I en la que se originan.

Tabla 9
Catálogo de casos de uso

Código	Caso de uso	Actor	Func.	Origen en el prototipo
CU-01	Registrarse	Usuario	F1	Requisito de la cuenta
CU-02	Iniciar sesión	Usuario	F1	Pantalla de carga e Inicio
CU-03	Cerrar sesión	Usuario	F1	Perfil: Cerrar sesión
CU-04	Gestionar perfil y configuración	Usuario	F1	Perfil: Configuración
CU-05	Crear meta de ahorro	Usuario	F2	Mis Metas: + Nueva
CU-06	Consultar metas de ahorro	Usuario	F2	Mis Metas e Inicio
CU-07	Administrar meta (pausar, reanudar, editar)	Usuario	F2	Mis Metas: menú de la meta
CU-08	Registrar ingreso o gasto	Usuario	F3	Inicio: Registrar gasto
CU-09	Depositar en meta de ahorro	Usuario	F3	Inicio: Abonar a meta
CU-10	Retirar de meta de ahorro	Usuario	F3	Fase I: depósitos y retiros
CU-11	Consultar historial de movimientos	Usuario	F3	Inicio: Ver movimientos
CU-12	Consultar lecciones por categoría	Usuario	F4	Aprende: Explorar temas
CU-13	Realizar lección y registrar progreso	Usuario	F4	Aprende: Continúa aprendiendo
CU-14	Consultar logros	Usuario	F4	Aprende: Tus logros
CU-15	Sincronizar datos (modo sin conexión)	Usuario	F5	Perfil: Modo sin conexión
CU-16	Administrar contenido educativo	Administrador	F4	Catálogo de Aprende
CU-17	Validar credenciales (incluido en CU-02)	Sistema	F1	—
CU-18	Bloquear cuenta temporalmente (extiende CU-02)	Sistema	F1	—
CU-19	Registrar transacción (incluido en CU-08, CU-09 y CU-10)	Sistema	F3	—
CU-20	Validar saldo suficiente (incluido en CU-10)	Sistema	F3	—
CU-21	Otorgar logro (extiende CU-13 y CU-19)	Sistema	F4	Aprende: Tus logros

Nota. Elaboración propia.

5.4 Descripción de los principales casos de uso

Se describen los seis casos de uso más representativos: iniciar sesión (CU-02), crear meta (CU-05), depositar en una meta (CU-09), retirar de una meta (CU-10), realizar una lección (CU-13) y sincronizar datos (CU-15). Cada descripción incluye nombre, actor principal, objetivo,

precondiciones, flujo principal y resultado esperado, y se complementa con los flujos alternos que los hacen coherentes con el modelo de datos.

Tabla 10
Descripción del caso de uso CU-02 Iniciar sesión

Elemento	Descripción
Nombre	CU-02 Iniciar sesión
Actor principal	Usuario
Objetivo	Autenticarse para acceder a sus datos y operaciones.
Precondiciones	1. El usuario tiene una cuenta activa (usuario.activo = 1). 2. El dispositivo tiene conexión con la API.
Flujo principal	1. El usuario ingresa su correo y su contraseña. 2. El sistema valida el formato de los datos y ejecuta CU-17 Validar credenciales: busca la cuenta y compara la contraseña con hash_contrasena. 3. El sistema verifica que la cuenta no esté bloqueada (bloqueado_hasta). 4. El sistema reinicia intentos_fallidos, actualiza ultimo_acceso y crea una fila en sesion con el hash del token de refresco. 5. El sistema entrega un token de acceso de corta duración y un token de refresco.
Flujos alternos	A1. Credenciales incorrectas: se incrementa intentos_fallidos y se informa un mensaje genérico que no revela si el correo existe. A2. Al superar el número de intentos permitido se ejecuta CU-18 y se fija bloqueado_hasta. A3. Cuenta bloqueada o inactiva: se rechaza el acceso.
Resultado esperado	Sesión creada y usuario autenticado; el token identifica al usuario en las demás operaciones.

Nota. Elaboración propia.

Tabla 11

Descripción del caso de uso CU-05 Crear meta de ahorro

Elemento	Descripción
Nombre	CU-05 Crear meta de ahorro
Actor principal	Usuario
Objetivo	Definir un objetivo de ahorro con su monto y fecha límite.
Precondiciones	Sesión activa.
Flujo principal	1. El usuario selecciona «+ Nueva» en la pantalla Mis Metas. 2. Ingresa nombre, descripción opcional, ícono, monto objetivo y fecha límite. 3. La aplicación valida los datos (monto mayor que cero y fecha no anterior a hoy). 4. El sistema genera el codigo_global e inserta la meta con monto_ahorrado = 0 y estado EN_PROGRESO. 5. Si es la primera meta del usuario, se ejecuta CU-21 para otorgar el logro «Primera meta». 6. La meta aparece en la lista de Mis Metas.
Flujos alternos	A1. Datos inválidos: se muestran los errores sin guardar. A2. Sin conexión: la meta se guarda en SQLite con su código global y se encola para CU-15.
Resultado esperado	Meta creada y visible en Mis Metas con un avance del 0 %.

Nota. Elaboración propia.

Tabla 12

Descripción del caso de uso CU-09 Depositar en meta de ahorro

Elemento	Descripción
Nombre	CU-09 Depositar en meta de ahorro
Actor principal	Usuario
Objetivo	Aumentar el saldo de una meta registrando un abono.
Precondiciones	1. Sesión activa. 2. La meta existe, pertenece al usuario y no está COMPLETADA ni PAUSADA.
Flujo principal	1. El usuario elige «Abonar a meta», selecciona la meta e ingresa el monto. 2. La aplicación valida que el monto sea mayor que cero. 3. El sistema inicia una transacción de base de datos. 4. Ejecuta CU-19 Registrar transacción: inserta un movimiento DEPOSITO_META con el monto y el codigo_global. 5. Incrementa meta_ahorro.monto_ahorrado y actualiza fecha_actualizacion. 6. Si el saldo alcanza el monto objetivo, cambia el estado a COMPLETADA. 7. Confirma la transacción (COMMIT). 8. Ejecuta CU-21 si corresponde un logro (por ejemplo, «Meta cumplida»).
Flujos alternos	A1. Si falla cualquier paso se revierte todo (ROLLBACK) y el saldo no cambia. A2. Si la meta pertenece a otro usuario, la clave foránea compuesta rechaza el movimiento. A3. Sin conexión: el movimiento se guarda en SQLite y se sincroniza después; si se reenvía, el código global evita duplicarlo.
Resultado esperado	Movimiento registrado y saldo de la meta actualizado de forma atómica; el porcentaje de avance refleja el nuevo saldo.

Nota. Elaboración propia.

Tabla 13

Descripción del caso de uso CU-10 Retirar de meta de ahorro

Elemento	Descripción
Nombre	CU-10 Retirar de meta de ahorro
Actor principal	Usuario
Objetivo	Disminuir el saldo de una meta cuando el usuario necesita usar parte del ahorro.
Precondiciones	1. Sesión activa. 2. La meta existe, pertenece al usuario y tiene saldo.
Flujo principal	1. El usuario selecciona la meta e ingresa el monto a retirar. 2. La aplicación valida que el monto sea mayor que cero. 3. El sistema inicia una transacción y bloquea la fila de la meta. 4. Ejecuta CU-20 Validar saldo suficiente: comprueba que el monto no supere monto_ahorrado. 5. Ejecuta CU-19 Registrar transacción: inserta un movimiento RETIRO_META. 6. Disminuye monto_ahorrado; si la meta estaba COMPLETADA y el saldo queda por debajo del objetivo, vuelve a EN_PROGRESO. 7. Confirma la transacción (COMMIT).
Flujos alternos	A1. Saldo insuficiente: el sistema rechaza el retiro y no registra nada. A2. Dos retiros simultáneos: el bloqueo de la fila los ordena; además, la restricción monto_ahorrado ≥ 0 impide un saldo negativo aunque falle la validación del código.
Resultado esperado	Retiro registrado y saldo disminuido sin llegar a ser negativo.

Nota. Elaboración propia.

Tabla 14

Descripción del caso de uso CU-13 Realizar lección y registrar progreso

Elemento	Descripción
Nombre	CU-13 Realizar lección y registrar progreso
Actor principal	Usuario
Objetivo	Estudiar una lección y guardar el avance para retomarla después.
Precondiciones	1. Sesión activa. 2. La lección está activa.
Flujo principal	1. El usuario abre una lección desde Explorar temas, Lección del día o Continúa aprendiendo. 2. El sistema muestra su contenido. 3. Al avanzar, la aplicación envía el porcentaje alcanzado. 4. El sistema crea o actualiza la fila de progreso_leccion (porcentaje y fecha_ultima_actividad). 5. Cuando el porcentaje llega a 100, marca completada y registra fecha_completado. 6. Ejecuta CU-21, que evalúa los logros (por ejemplo, «3 lecciones»).
Flujos alternos	A1. Si el porcentaje recibido es menor que el ya guardado, se conserva el mayor. A2. Una lección inactiva no se ofrece al usuario.
Resultado esperado	Progreso guardado (único por usuario y lección) y contador de lecciones del panel actualizado.

Nota. Elaboración propia.

Tabla 15
Descripción del caso de uso CU-15 Sincronizar datos

Elemento	Descripción
Nombre	CU-15 Sincronizar datos (modo sin conexión)
Actor principal	Usuario (el sistema la inicia automáticamente al recuperar la conexión)
Objetivo	Enviar al servidor las operaciones registradas sin conexión y recibir los cambios pendientes.
Precondiciones	1. Sesión válida. 2. Hay operaciones en la cola local. 3. Hay conexión con la API.
Flujo principal	1. La aplicación detecta la conexión y toma las operaciones pendientes de la cola. 2. Envía un lote a la API con el identificador del dispositivo. 3. El sistema crea una fila en sincronizacion con estado EN_CURSO. 4. Por cada operación, en su propia transacción: si su codigo_global ya existe, la descarta; si no, la aplica (crear meta o registrar movimiento). 5. Actualiza operaciones_recibidas y operaciones_aplicadas. 6. Cierra la sincronización como COMPLETADA, PARCIAL o FALLIDA y responde el resultado por operación. 7. La aplicación marca como enviadas las operaciones aplicadas.
Flujos alternos	A1. Se interrumpe la conexión: la sincronización queda FALLIDA con detalle_error y las operaciones siguen en la cola para reintentarse. A2. Una operación es inválida: se rechaza solo esa operación y la sincronización queda PARCIAL.
Resultado esperado	Datos locales y del servidor coinciden; cada operación se aplicó exactamente una vez.

Nota. Elaboración propia.

6. Diagrama de clases UML

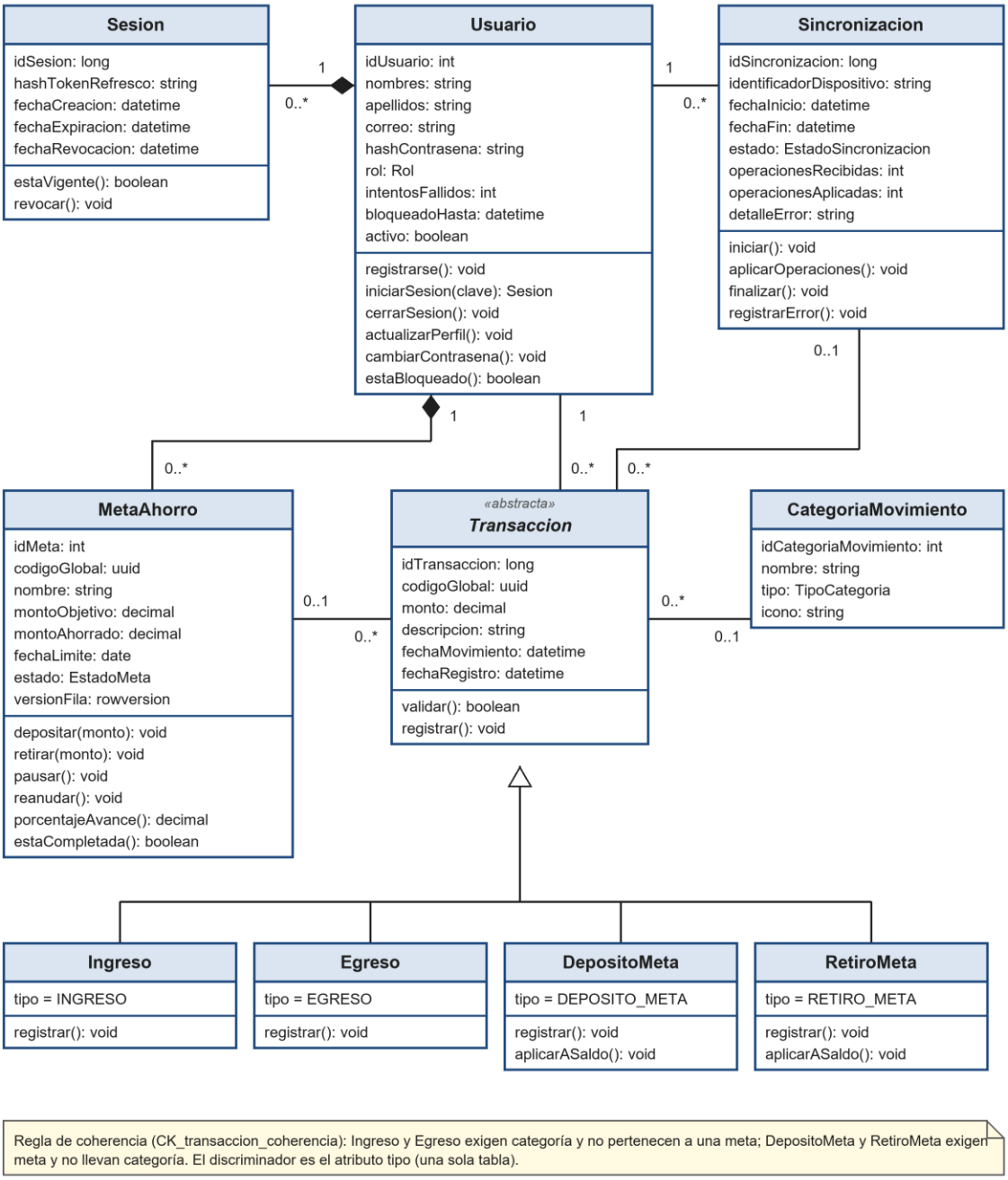
6.1 Criterios de modelado

El diagrama de clases representa la estructura estática del sistema y se derivó de los casos de uso: cada funcionalidad cuenta con las clases necesarias para soportarla. Se modelaron las clases de dominio, es decir, las que representan conceptos que el sistema debe recordar, con sus atributos principales y las operaciones que resuelven los casos de uso. Por legibilidad el modelo se presenta en dos vistas que comparten la clase Usuario: la vista financiera y de seguridad (Figura 3) y la vista educativa (Figura 4). La lista completa de campos de cada clase está en el diccionario de datos de la sección 7.3.

6.2 Vista financiera y de seguridad

Figura 3

Diagrama de clases: vista financiera y de seguridad

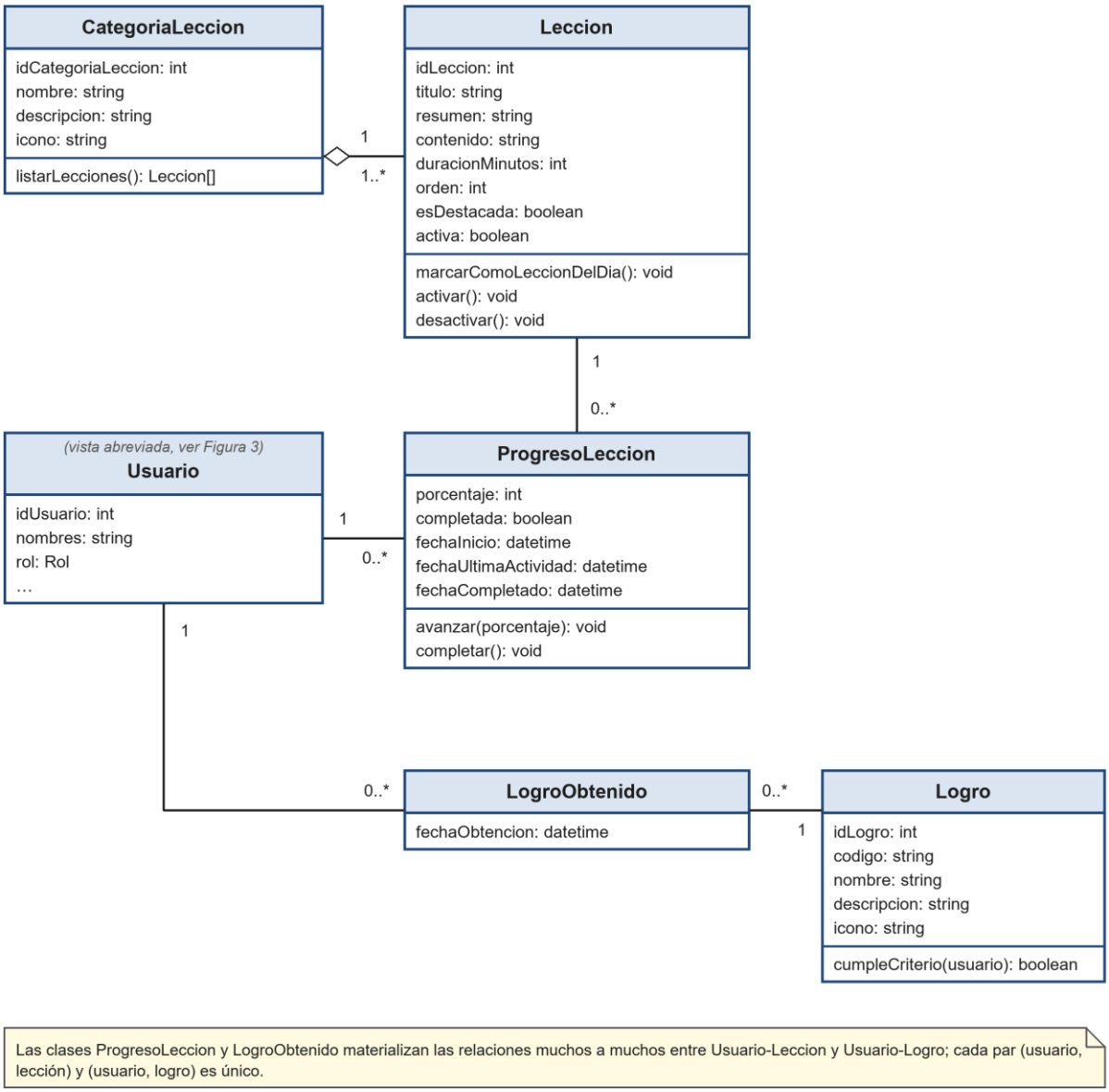


Nota. Elaboración propia con notación UML 2.5.1. Se muestran los atributos principales; la lista completa está en la sección 7.3. Los rombos negros indican composición.

En esta vista, Usuario es la clase central: compone sus metas y sus sesiones (si el usuario se elimina, dejan de tener sentido) y se asocia con los movimientos y las sincronizaciones que realiza. Transaccion es una clase abstracta con cuatro especializaciones que comparten atributos y difieren en su comportamiento: los ingresos y egresos se clasifican con una CategoriaMovimiento, mientras que los depósitos y retiros afectan el saldo de una MetaAhorro. Las asociaciones opcionales de Transaccion (multiplicidad 0..1) hacia MetaAhorro, CategoriaMovimiento y Sincronizacion reflejan esa distinción y la regla de coherencia que indica la nota del diagrama.

6.3 Vista educativa

Figura 4
Diagrama de clases: vista educativa



Nota. Elaboración propia con notación UML 2.5.1. El rombo blanco indica agregación; Usuario se muestra de forma abreviada.

La vista educativa modela la pantalla Aprende. Una CategoriaLeccion agrega varias Leccion (agregación: las lecciones podrían reasignarse a otra categoría). El avance de cada usuario en

cada lección se representa con ProgresoLeccion y los logros conseguidos con LogroObtenido; ambas clases resuelven relaciones de muchos a muchos con atributos propios.

6.4 Descripción de las clases

La Tabla 16 resume la responsabilidad y las operaciones principales de cada clase y la tabla del modelo relacional en que se almacena. Las enumeraciones Rol (USUARIO, ADMINISTRADOR), EstadoMeta (EN_PROGRESO, COMPLETADA, PAUSADA, ATRASADA), TipoCategoria (INGRESO, EGRESO) y EstadoSincronizacion (EN_CURSO, COMPLETADA, PARCIAL, FALLIDA) se implementan como restricciones CHECK.

Tabla 16
Clases del sistema, responsabilidades y tabla asociada

Clase	Responsabilidad	Operaciones principales	Tabla
Usuario	Representa a la persona que usa el sistema y gestiona su cuenta y su acceso.	registrarse, iniciarSesion, cerrarSesion, actualizarPerfil, cambiarContrasena, estaBloqueado	usuario
Sesion	Representa una sesión iniciada y permite revocarla.	estaVigente, revocar	sesion
MetaAhorro	Guarda un objetivo de ahorro y su saldo; aplica depósitos y retiros.	depositar, retirar, pausar, reanudar, porcentajeAvance, estaCompletada	meta_ahorro
Transaccion (abstracta)	Movimiento financiero con monto, descripción y fechas.	validar, registrar	transaccion
Ingreso y Egreso	Movimientos que entran o salen del presupuesto del usuario, clasificados con una categoría.	registrar	transaccion (tipo INGRESO o EGRESO)
DepositoMeta y RetiroMeta	Movimientos que aumentan o disminuyen el saldo de una meta.	registrar, aplicarASaldo	transaccion (tipo DEPOSITO_META o RETIRO_META)
CategoriaMovimiento	Clasifica los ingresos y los egresos.	—	categoria_movimiento
Sincronizacion	Registra cada intento de sincronización de un dispositivo.	iniciar, aplicarOperaciones, finalizar, registrarError	sincronizacion
CategoriaLeccion	Agrupa las lecciones por tema.	listarLecciones	categoria_leccion
Leccion	Contenido educativo corto; una de ellas puede ser la lección del día.	marcarComoLeccionDelDia, activar, desactivar	leccion
ProgresoLeccion	Avance de un usuario en una lección.	avanzar, completar	progreso_leccion
Logro	Insignia con su criterio de obtención.	cumpleCriterio	logro
LogroObtenido	Registra que un usuario obtuvo un logro y cuándo.	—	usuario_logro

Nota. Elaboración propia.

6.5 Relaciones y multiplicidades

La Tabla 17 recoge las relaciones del diagrama. La multiplicidad se lee de izquierda a derecha: el primer valor indica cuántas instancias de la primera clase participan por cada instancia de la segunda, y el segundo valor, a la inversa.

Tabla 17
Relaciones entre clases

Relación	Tipo	Multiplicidad	Justificación
Usuario y Sesion	Composición	1 y 0..*	Una sesión no existe sin su usuario.
Usuario y MetaAhorro	Composición	1 y 0..*	Las metas pertenecen a un único usuario y no tienen sentido sin él.
Usuario y Transaccion	Asociación	1 y 0..*	El usuario registra muchos movimientos; cada movimiento es de un solo usuario.
Usuario y Sincronizacion	Asociación	1 y 0..*	Cada dispositivo del usuario sincroniza muchas veces.
MetaAhorro y Transaccion	Asociación	0..1 y 0..*	Solo los depósitos y retiros pertenecen a una meta.
CategoriaMovimiento y Transaccion	Asociación	0..1 y 0..*	Solo los ingresos y egresos llevan categoría.
Sincronizacion y Transaccion	Asociación	0..1 y 0..*	Los movimientos creados sin conexión referencian la sincronización que los aplicó.
Transaccion, Ingreso, Egreso, DepositoMeta y RetiroMeta	Generalización	—	Cuatro tipos de movimiento con atributos comunes.
CategoriaLeccion y Leccion	Agregación	1 y 1..*	Una categoría agrupa lecciones que existen por sí mismas.
Usuario, ProgresoLeccion y Leccion	Asociación	1 y 0..*; 1 y 0..*	Relación de muchos a muchos con atributos propios.
Usuario, LogroObtenido y Logro	Asociación	1 y 0..*; 1 y 0..*	Relación de muchos a muchos con atributos propios.

Nota. Elaboración propia.

6.6 Coherencia con los casos de uso

Cada caso de uso se apoya en clases concretas del diagrama (Tabla 18). Esta verificación asegura que no exista funcionalidad sin clases que la soporten ni clases sin uso.

Tabla 18
Correspondencia entre casos de uso y clases

Caso de uso	Clases que participan	Operaciones clave
CU-01 Registrarse	Usuario	registrarse
CU-02 Iniciar sesión (CU-17, CU-18)	Usuario, Sesion	iniciarSesion, estaBloqueado

Caso de uso	Clases que participan	Operaciones clave
CU-03 Cerrar sesión	Usuario, Sesion	cerrarSesion, revocar
CU-04 Gestionar perfil y configuración	Usuario	actualizarPerfil, cambiarContrasena
CU-05 Crear meta de ahorro	Usuario, MetaAhorro	creación de la meta
CU-06 Consultar metas de ahorro	MetaAhorro	porcentajeAvance
CU-07 Administrar meta	MetaAhorro	pausar, reanudar, estaCompletada
CU-08 Registrar ingreso o gasto (CU-19)	Ingreso, Egreso, CategoriaMovimiento	validar, registrar
CU-09 Depositar en meta (CU-19)	DepositoMeta, MetaAhorro	registrar, aplicarASaldo, depositar
CU-10 Retirar de meta (CU-19, CU-20)	RetiroMeta, MetaAhorro	registrar, aplicarASaldo, retirar
CU-11 Consultar historial de movimientos	Transaccion y sus especializaciones, CategoriaMovimiento	consulta de movimientos
CU-12 Consultar lecciones por categoría	CategoriaLeccion, Leccion	listarLecciones
CU-13 Realizar lección y registrar progreso	Leccion, ProgresoLeccion	avanzar, completar
CU-14 Consultar logros	Logro, LogroObtenido	consulta de logros
CU-15 Sincronizar datos	Sincronizacion, Transaccion, MetaAhorro	iniciar, aplicarOperaciones, finalizar
CU-16 Administrar contenido educativo	CategoriaLeccion, Leccion, Logro	activar, desactivar, marcarComoLeccionDelDia
CU-21 Otorgar logro	Logro, LogroObtenido	cumpleCriterio

Nota. Elaboración propia.

7. Modelo relacional

7.1 Criterios de diseño

El modelo relacional traduce las clases persistentes del diagrama a tablas siguiendo el modelo de Codd (1970) y las reglas de correspondencia entre el modelo de clases y el modelo relacional descritas por Elmasri y Navathe (2016). La Tabla 19 resume los criterios aplicados.

Tabla 19
Criterios de diseño del modelo relacional

Aspecto	Decisión	Motivo
Nombres	Nombres en snake_case y en español; tablas en singular; restricciones nombradas con los prefijos PK_, FK_, UQ_, CK_ y DF_.	Legibilidad y mensajes de error que identifican la regla incumplida.
Claves primarias	Identificadores autogenerados (INT o BIGINT con IDENTITY) y claves compuestas en las tablas de asociación.	Simplicidad y estabilidad de las referencias.
Identificador global	Columna codigo_global (UNIQUEIDENTIFIER) en meta_ahorro y transaccion.	Crear registros sin conexión y evitar duplicados al sincronizar.
Tipos de datos	DECIMAL(14,2) para el dinero, DATETIME2 en UTC para las fechas, NVARCHAR para el texto con tildes y BIT para las banderas.	Precisión exacta en los montos y soporte del español.
Dominios	Las enumeraciones se implementan como restricciones CHECK.	La propia base de datos aplica los valores válidos.
Herencia	Una sola tabla transaccion con el discriminador tipo.	Las cuatro especializaciones comparten casi todos los atributos y el historial se consulta sin uniones.
Relaciones de muchos a muchos	Tablas progreso_leccion y usuario_logro con clave primaria compuesta.	Permiten guardar atributos propios de la relación.
Borrado	Sin ON DELETE CASCADE; baja lógica en usuario y leccion.	Los datos financieros no deben desaparecer por un borrado accidental.
Concurrencia	Columna ROWVERSION en meta_ahorro.	Control de concurrencia optimista al editar una meta.

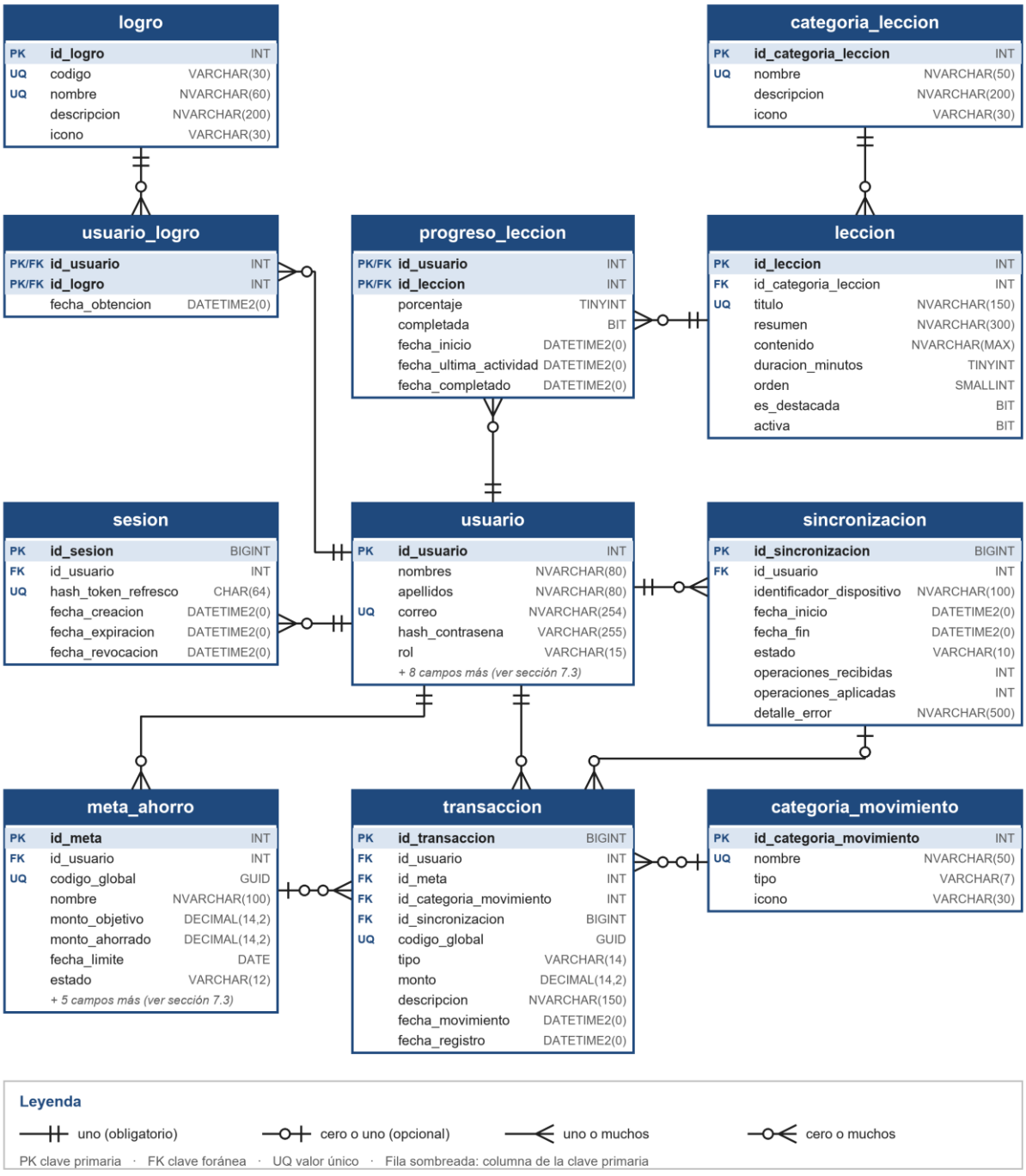
Nota. Elaboración propia.

7.2 Diagrama del modelo relacional

La Figura 5 muestra el modelo relacional con la notación de pie de gallo. El diagrama se generó a partir del catálogo de la base de datos implementada, de modo que coincide con ella. Solo se omiten, por espacio, los campos menos relevantes de usuario (8) y meta_ahorro (5), que sí aparecen en el diccionario de datos.

Figura 5

Modelo relacional de GKMF



Nota. Elaboración propia a partir del catálogo de la base de datos GKMF (SQL Server).

7.3 Diccionario de datos

El modelo contiene 11 tablas y 85 campos. Las tablas 20 a 30 describen cada campo con su tipo de dato, si admite nulos y sus claves. La columna Clave usa PK (clave primaria), FK (clave foránea) y UQ (valor único). Cuando existe, el valor por defecto se indica al final de la descripción.

Tabla 20

Diccionario de datos de la tabla usuario

Campo	Tipo de dato	Nulo	Clave	Descripción
id_usuario	INT	No	PK	Identificador único autogenerado del usuario.
nombres	NVARCHAR(80)	No		Nombres de la persona.
apellidos	NVARCHAR(80)	No		Apellidos de la persona.
correo	NVARCHAR(254)	No	UQ	Correo electrónico; identifica al usuario al iniciar sesión (único, sin distinguir mayúsculas).
hash_contrasena	VARCHAR(255)	No		Hash con sal de la contraseña (scrypt); nunca se guarda la contraseña en claro.
rol	VARCHAR(15)	No		Rol de acceso: USUARIO o ADMINISTRADOR. Valor por defecto: USUARIO.
idioma	CHAR(2)	No		Idioma de la interfaz: es o en. Valor por defecto: es.
notificaciones_activas	BIT	No		Indica si el usuario desea recibir notificaciones. Valor por defecto: 1.
modo_offline_activo	BIT	No		Indica si el modo sin conexión está habilitado. Valor por defecto: 1.
intentos_fallidos	TINYINT	No		Inicios de sesión fallidos consecutivos (protección contra fuerza bruta). Valor por defecto: 0.
bloqueado_hasta	DATETIME2(0)	Sí		Fecha y hora hasta la que la cuenta permanece bloqueada; nulo si no lo está.
ultimo_acceso	DATETIME2(0)	Sí		Fecha y hora (UTC) del último inicio de sesión exitoso.
activo	BIT	No		Baja lógica: 0 desactiva la cuenta sin borrar sus datos financieros. Valor por defecto: 1.
fecha_registro	DATETIME2(0)	No		Fecha y hora (UTC) en que se creó la cuenta. Valor por defecto: fecha y hora UTC actual.

Nota. Persona registrada en la aplicación (ahorrador o administrador de contenido). Clase asociada: Usuario. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 21

Diccionario de datos de la tabla sesion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_sesion	BIGINT	No	PK	Identificador único autogenerado de la sesión.
id_usuario	INT	No	FK	Usuario dueño de la sesión.
hash_token_refresco	CHAR(64)	No	UQ	Hash SHA-256 (64 caracteres hexadecimales) del token de refresco; nunca el token.
fecha_creacion	DATETIME2(0)	No		Fecha y hora (UTC) de inicio de la sesión. Valor por defecto: fecha y hora UTC actual.
fecha_expiracion	DATETIME2(0)	No		Fecha y hora (UTC) en que la sesión deja de ser válida.
fecha_revocacion	DATETIME2(0)	Sí		Fecha y hora (UTC) del cierre de sesión; nulo mientras siga activa.

Nota. Sesión iniciada por un usuario; guarda solo el hash del token de refresco. Clase asociada: Sesion. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 22

Diccionario de datos de la tabla meta_ahorro

Campo	Tipo de dato	Nulo	Clave	Descripción
id_meta	INT	No	PK	Identificador único autogenerado de la meta.
id_usuario	INT	No	FK	Usuario propietario de la meta.
codigo_global	UNIQUEIDENTIFIER	No	UQ	GUID generado en el dispositivo; permite crear la meta sin conexión sin duplicados. Valor por defecto: GUID aleatorio (NEWID).
nombre	NVARCHAR(100)	No		Nombre de la meta (por ejemplo, Mercado del mes).
descripcion	NVARCHAR(250)	Sí		Descripción opcional de la meta.
icono	VARCHAR(30)	Sí		Clave del ícono que muestra la interfaz.
monto_objetivo	DECIMAL(14,2)	No		Valor que se desea ahorrar; debe ser mayor que cero.
monto_ahorrado	DECIMAL(14,2)	No		Saldo actual de la meta (depósitos menos retiros); nunca es negativo. Valor por defecto: 0.
fecha_limite	DATE	No		Fecha en que se espera alcanzar el objetivo.
estado	VARCHAR(12)	No		Estado: EN_PROGRESO, COMPLETADA, PAUSADA o ATRASADA. Valor por defecto: EN_PROGRESO.
fecha_creacion	DATETIME2(0)	No		Fecha y hora (UTC) de creación de la meta. Valor por defecto: fecha y hora UTC actual.
fecha_actualizacion	DATETIME2(0)	No		Fecha y hora (UTC) de la última modificación (resolución de conflictos al sincronizar). Valor por defecto: fecha y hora UTC actual.
version_fila	ROWVERSION	No		Marca de versión automática de la fila (control de concurrencia optimista).

Nota. Objetivo de ahorro de un usuario, con su saldo, fecha límite y estado. Clase asociada: MetaAhorro. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 23

Diccionario de datos de la tabla categoria_movimiento

Campo	Tipo de dato	Nulo	Clave	Descripción
id_categoria_movimiento	INT	No	PK	Identificador único autogenerado de la categoría.
nombre	NVARCHAR(50)	No	UQ	Nombre de la categoría (Alimentación, Transporte, etc.); único.
tipo	VARCHAR(7)	No		Naturaleza de la categoría: INGRESO o EGRESO.
icono	VARCHAR(30)	Sí		Clave del ícono que muestra la interfaz.

Nota. Catálogo de categorías para clasificar ingresos y egresos. Clase asociada: CategoriaMovimiento. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 24

Diccionario de datos de la tabla transaccion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_transaccion	BIGINT	No	PK	Identificador único autogenerado del movimiento.
id_usuario	INT	No	FK	Usuario que realizó el movimiento.
id_meta	INT	Sí	FK	Meta afectada; solo en depósitos y retiros de meta.
id_categoria_movimiento	INT	Sí	FK	Categoría del movimiento; solo en ingresos y egresos.
id_sincronizacion	BIGINT	Sí	FK	Sincronización que aplicó el movimiento; nulo si se registró en línea.
codigo_global	UNIQUEIDENTIFIER	No	UQ	GUID generado en el dispositivo; garantiza que un movimiento sincronizado dos veces se aplique una sola vez. Valor por defecto: GUID aleatorio (NEWID).
tipo	VARCHAR(14)	No		Tipo de movimiento: INGRESO, EGRESO, DEPOSITO_META o RETIRO_META.
monto	DECIMAL(14,2)	No		Valor del movimiento; siempre mayor que cero (el signo lo da el tipo).
descripcion	NVARCHAR(150)	No		Texto que describe el movimiento (por ejemplo, Bus SITP).
fecha_movimiento	DATETIME2(0)	No		Fecha y hora (UTC) en que ocurrió el movimiento, según el dispositivo. Valor por defecto: fecha y hora UTC actual.
fecha_registro	DATETIME2(0)	No		Fecha y hora (UTC) en que el servidor lo registró. Valor por defecto: fecha y hora UTC actual.

Nota. Movimiento financiero: ingreso, egreso, depósito o retiro de una meta. Clase asociada: Transaccion (Ingreso, Egreso, DepositoMeta, RetiroMeta). Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 25

Diccionario de datos de la tabla sincronizacion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_sincronizacion	BIGINT	No	PK	Identificador único autogenerado de la sincronización.
id_usuario	INT	No	FK	Usuario cuyo dispositivo sincroniza.
identificador_dispositivo	NVARCHAR(100)	No		Identificador del dispositivo que envía las operaciones.
fecha_inicio	DATETIME2(0)	No		Fecha y hora (UTC) en que comenzó la sincronización. Valor por defecto: fecha y hora UTC actual.
fecha_fin	DATETIME2(0)	Sí		Fecha y hora (UTC) en que terminó; nulo mientras esté en curso.
estado	VARCHAR(10)	No		Estado: EN_CURSO, COMPLETADA, PARCIAL o FALLIDA. Valor por defecto: EN_CURSO.
operaciones_recibidas	INT	No		Cantidad de operaciones pendientes que envió el dispositivo. Valor por defecto: 0.
operaciones_aplicadas	INT	No		Cantidad de operaciones que el servidor aplicó; no supera las recibidas. Valor por defecto: 0.
detalle_error	NVARCHAR(500)	Sí		Descripción del error cuando la sincronización falla o es parcial.

Nota. Sesión de sincronización entre el dispositivo y el servidor. Clase asociada: Sincronizacion. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 26

Diccionario de datos de la tabla categoria_leccion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_categoria_leccion	INT	No	PK	Identificador único autogenerado de la categoría.
nombre	NVARCHAR(50)	No	UQ	Nombre del tema (Ahorro, Crédito, Presupuesto, Protección, Inversión, Familia); único.
descripcion	NVARCHAR(200)	Sí		Descripción opcional del tema.
icono	VARCHAR(30)	Sí		Clave del ícono que muestra la interfaz.

Nota. Temas de educación financiera. Clase asociada: CategoriaLeccion. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 27

Diccionario de datos de la tabla leccion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_leccion	INT	No	PK	Identificador único autogenerado de la lección.
id_categoria_leccion	INT	No	FK	Categoría a la que pertenece la lección.
titulo	NVARCHAR(150)	No	UQ	Título de la lección; único.
resumen	NVARCHAR(300)	No		Resumen breve que se muestra en la lista y en la lección del día.
contenido	NVARCHAR(MAX)	Sí		Cuerpo de la lección; opcional.
duracion_minutos	TINYINT	No		Duración estimada de lectura, entre 1 y 120 minutos.
orden	SMALLINT	No		Posición de la lección dentro de su categoría; mayor que cero. Valor por defecto: 1.
es_destacada	BIT	No		Marca la lección del día; solo una lección puede estarlo a la vez. Valor por defecto: 0.
activa	BIT	No		Indica si la lección está publicada. Valor por defecto: 1.

Nota. Lección corta de educación financiera. Clase asociada: Leccion. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 28

Diccionario de datos de la tabla progreso_leccion

Campo	Tipo de dato	Nulo	Clave	Descripción
id_usuario	INT	No	PK, FK	Usuario que cursa la lección (parte de la clave primaria).
id_leccion	INT	No	PK, FK	Lección cursada (parte de la clave primaria).
porcentaje	TINYINT	No		Avance de 0 a 100. Valor por defecto: 0.
completada	BIT	No		Indica si el usuario terminó la lección. Valor por defecto: 0.
fecha_inicio	DATETIME2(0)	No		Fecha y hora (UTC) en que el usuario empezó la lección. Valor por defecto: fecha y hora UTC actual.
fecha_ultima_actividad	DATETIME2(0)	No		Fecha y hora (UTC) de la última interacción. Valor por defecto: fecha y hora UTC actual.
fecha_completado	DATETIME2(0)	Sí		Fecha y hora (UTC) de finalización; solo si está completada.

Nota. Avance de un usuario en una lección. Clase asociada: ProgresoLeccion. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 29

Diccionario de datos de la tabla logro

Campo	Tipo de dato	Nulo	Clave	Descripción
id_logro	INT	No	PK	Identificador único autogenerado del logro.
codigo	VARCHAR(30)	No	UQ	Código estable del logro (por ejemplo, PRIMERA_META); único.
nombre	NVARCHAR(60)	No	UQ	Nombre visible del logro; único.
descripcion	NVARCHAR(200)	No		Condición para obtener el logro.
icono	VARCHAR(30)	Sí		Clave del ícono que muestra la interfaz.

Nota. Catálogo de logros (insignias) que se pueden obtener. Clase asociada: Logro. Elaboración propia a partir del catálogo de la base de datos GKMF.

Tabla 30

Diccionario de datos de la tabla usuario_logro

Campo	Tipo de dato	Nulo	Clave	Descripción
id_usuario	INT	No	PK, FK	Usuario que obtuvo el logro (parte de la clave primaria).
id_logro	INT	No	PK, FK	Logro obtenido (parte de la clave primaria).
fecha_obtencion	DATETIME2(0)	No		Fecha y hora (UTC) en que se obtuvo el logro. Valor por defecto: fecha y hora UTC actual.

Nota. Logro obtenido por un usuario. Clase asociada: LogroObtenido. Elaboración propia a partir del catálogo de la base de datos GKMF.

7.4 Relaciones y cardinalidades

La Tabla 31 lista las 12 claves foráneas. La cardinalidad se lee de la tabla padre a la hija; las claves foráneas opcionales, que admiten nulo, permiten que un movimiento pertenezca o no a una meta, a una categoría o a una sincronización. Dos claves foráneas son compuestas: (id_meta, id_usuario) e (id_sincronizacion, id_usuario). Al incluir id_usuario en la referencia, la propia base de datos impide que un movimiento del usuario A apunte a una meta o a una sincronización del usuario B. Ninguna clave foránea elimina en cascada.

Tabla 31
Claves foráneas, relaciones y cardinalidades

Clave foránea	Tabla hija (columnas)	Tabla padre (columnas)	Cardinalidad	Obligatoria
FK_sesion_usuario	sesion (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí
FK_meta_ahorro_usuario	meta_ahorro (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí
FK_transaccion_categoria	transaccion (id_categoria_movimiento)	categoria_movimiento (id_categoria_movimiento)	0..1 a 0..N	No
FK_transaccion_meta	transaccion (id_meta, id_usuario)	meta_ahorro (id_meta, id_usuario)	0..1 a 0..N	No
FK_transaccion_sincronizacion	transaccion (id_sincronizacion, id_usuario)	sincronizacion (id_sincronizacion, id_usuario)	0..1 a 0..N	No
FK_transaccion_usuario	transaccion (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí
FK_sincronizacion_usuario	sincronizacion (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí
FK_leccion_categoria	leccion (id_categoria_leccion)	categoria_leccion (id_categoria_leccion)	1 a 0..N	Sí
FK_progreso_leccion_leccion	progreso_leccion (id_leccion)	leccion (id_leccion)	1 a 0..N	Sí
FK_progreso_leccion_usuario	progreso_leccion (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí
FK_usuario_logro_logro	usuario_logro (id_logro)	logro (id_logro)	1 a 0..N	Sí
FK_usuario_logro_usuario	usuario_logro (id_usuario)	usuario (id_usuario)	1 a 0..N	Sí

Nota. Elaboración propia a partir del catálogo de la base de datos GKMF. Todas las claves foráneas usan NO ACTION al borrar.

7.5 Restricciones de integridad

Además de las claves primarias y foráneas, el modelo define 11 restricciones de unicidad y 23 restricciones CHECK (Tabla 32), todas con nombre explícito. Las reglas se verifican en el propio motor y se comprobaron con 29 pruebas de datos inválidos, que la base de datos rechazó en su totalidad.

Tabla 32
Restricciones de unicidad (UQ) y de verificación (CK)

Tabla	Restricción	Regla que aplica
usuario	CK_usuario_correo_formato	El correo tiene formato válido (texto@dominio.ext) y no contiene espacios.
usuario	CK_usuario_hash_longitud	El hash tiene al menos 32 caracteres: impide guardar una contraseña en texto plano.
usuario	CK_usuario_idioma	El idioma solo puede ser es o en.
usuario	CK_usuario_rol	El rol solo puede ser USUARIO o ADMINISTRADOR.
usuario	UQ_usuario_correo	El correo electrónico no puede repetirse (sin distinguir mayúsculas).
sesion	CK_sesion_expiracion	La sesión expira después de haberse creado.

Tabla	Restricción	Regla que aplica
sesion	CK_sesion_hash_hexadecimal	El hash del token contiene solo caracteres hexadecimales.
sesion	CK_sesion_revocacion	La revocación no es anterior a la creación.
sesion	UQ_sesion_hash_token	Cada token de refresco tiene un hash distinto.
meta_ahorro	CK_meta_ahorro_ahorrado_no_negativo	El saldo ahorrado nunca es negativo.
meta_ahorro	CK_meta_ahorro_completada	Una meta COMPLETADA ya alcanzó su monto objetivo.
meta_ahorro	CK_meta_ahorro_estado	El estado es EN_PROGRESO, COMPLETADA, PAUSADA o ATRASADA.
meta_ahorro	CK_meta_ahorro_fecha_limite	La fecha límite no es anterior a la fecha de creación (se tolera un día por la diferencia entre la hora local y UTC).
meta_ahorro	CK_meta_ahorro_objetivo_positivo	El monto objetivo es mayor que cero.
meta_ahorro	UQ_meta_ahorro_codigo_global	El código global de una meta es único (evita duplicados al sincronizar).
meta_ahorro	UQ_meta_ahorro_id_usuario	Par (id_meta, id_usuario) único; es el destino de la clave foránea compuesta de transaccion.
categoria_movimiento	CK_categoria_movimiento_tipo	El tipo de categoría es INGRESO o EGRESO.
categoria_movimiento	UQ_categoria_movimiento_nombre	No puede haber dos categorías de movimiento con el mismo nombre.
transaccion	CK_transaccion_coherencia	Ingreso y egreso exigen categoría y no llevan meta; depósito y retiro exigen meta y no llevan categoría.
transaccion	CK_transaccion_monto_positivo	El monto es mayor que cero.
transaccion	CK_transaccion_tipo	El tipo es INGRESO, EGRESO, DEPOSITO_META o RETIRO_META.
transaccion	UQ_transaccion_codigo_global	Un movimiento sincronizado varias veces se registra una sola vez (idempotencia).
sincronizacion	CK_sincronizacion_estado	El estado es EN_CURSO, COMPLETADA, PARCIAL o FALLIDA.
sincronizacion	CK_sincronizacion_fin	Solo una sincronización EN_CURSO carece de fecha de fin, y la fecha de fin no precede al inicio.
sincronizacion	CK_sincronizacion_operaciones	Las operaciones aplicadas no superan las recibidas y ninguna cuenta es negativa.
sincronizacion	UQ_sincronizacion_id_usuario	Par (id_sincronizacion, id_usuario) único; es el destino de la clave foránea compuesta de transaccion.
categoria_leccion	UQ_categoria_leccion_nombre	No puede haber dos categorías de lección con el mismo nombre.
leccion	CK_leccion_duracion	La duración está entre 1 y 120 minutos.
leccion	CK_leccion_orden	El orden es mayor que cero.
leccion	UQ_leccion_titulo	No puede haber dos lecciones con el mismo título.
progreso_leccion	CK_progreso_leccion_completada	Una lección completada tiene 100 % y fecha de finalización; una incompleta, menos de 100 % y sin fecha.
progreso_leccion	CK_progreso_leccion_porcentaje	El porcentaje está entre 0 y 100.
logro	UQ_logro_codigo	El código del logro es único.
logro	UQ_logro_nombre	El nombre del logro es único.

Nota. Elaboración propia a partir del catálogo de la base de datos GKMF. Los valores por defecto (DF_) se detallan en el diccionario de datos.

7.6 Índices

Además de los índices que crean las claves primarias y las restricciones UNIQUE, se definieron 11 índices para las consultas frecuentes de las pantallas del prototipo (Tabla 33). Tres de ellos son filtrados (solo indexan las filas que cumplen una condición) y uno garantiza que exista una única lección del día.

Tabla 33
Índices del modelo relacional

Índice	Tabla	Columnas	Propósito
IX_sesion_usuario	sesion	id_usuario, fecha_revocacion	Consulta de las sesiones activas de un usuario.
IX_meta_ahorro_usuario_estado	meta_ahorro	id_usuario, estado (incluye nombre, monto_objetivo, monto_ahorrado, fecha_limite)	Pantalla Mis Metas: metas de un usuario filtradas por estado (índice cubriente).
IX_transaccion_categoria	transaccion	id_categoria_movimiento — filtro [id_categoria_movimiento] IS NOT NULL	Movimientos de una categoría (índice filtrado: solo filas con categoría).
IX_transaccion_meta	transaccion	id_meta — filtro [id_meta] IS NOT NULL	Movimientos de una meta (índice filtrado: solo filas con meta).
IX_transaccion_sincronizacion	transaccion	id_sincronizacion — filtro [id_sincronizacion] IS NOT NULL	Movimientos aplicados por una sincronización (índice filtrado).
IX_transaccion_usuario_fecha	transaccion	id_usuario, fecha_movimiento (incluye tipo, monto, descripcion)	Historial de movimientos de un usuario, del más reciente al más antiguo (índice cubriente).
IX_sincronizacion_usuario_fecha	sincronizacion	id_usuario, fecha_inicio	Historial de sincronizaciones de un usuario, de la más reciente a la más antigua.
IX_leccion_categoria_orden	leccion	id_categoria_leccion, orden	Lecciones de una categoría en su orden de presentación.
UX_leccion_destacada	leccion	es_destacada — filtro [es_destacada]=(1	Garantiza que solo una lección sea la lección del día (índice único filtrado).
IX_progreso_leccion_leccion	progreso_leccion	id_leccion	Consulta del avance de todos los usuarios en una lección.
IX_usuario_logro_logro	usuario_logro	id_logro	Consulta de qué usuarios obtuvieron un logro.

Nota. Elaboración propia a partir del catálogo de la base de datos GKMF.

7.7 Correspondencia entre clases y tablas

La Tabla 34 muestra cómo cada elemento del diagrama de clases se transforma en un elemento del modelo relacional. Las 11 tablas provienen de 11 clases persistentes; la jerarquía de

Transaccion se implementa con una sola tabla, cuyo discriminador es la columna tipo. La correspondencia clase por clase aparece en la última columna de la Tabla 16.

Tabla 34
Correspondencia entre el modelo de clases y el modelo relacional

Modelo de clases	Modelo relacional
Clase persistente	Tabla con el mismo nombre en snake_case
Atributo	Columna con un tipo de dato SQL
Identificador de la clase	Clave primaria
Asociación con multiplicidad 1 a 0..*	Clave foránea NOT NULL en la tabla del lado «muchos»
Asociación opcional (0..1)	Clave foránea que admite nulo
Composición	Clave foránea NOT NULL sin borrado en cascada
Generalización de Transaccion	Una tabla con el discriminador tipo y la restricción CK_transaccion_coherencia
Clases ProgresoLeccion y LogroObtenido	Tablas con clave primaria compuesta (usuario y lección; usuario y logro)
Enumeración	Restricción CHECK con los valores permitidos
Regla de negocio de una operación (por ejemplo, retirar)	Servicio de negocio más restricción de respaldo (saldo mayor o igual que cero)

Nota. Elaboración propia.

7.8 Normalización

El modelo cumple la tercera forma normal (3FN): cada campo depende de la clave completa y solo de ella, y no hay dependencias transitivas. Por ejemplo, el nombre de una categoría se guarda una sola vez en categoria_movimiento y los movimientos solo la referencian (Elmasri y Navathe, 2016).

Existe una desnormalización deliberada: meta_ahorro.monto_ahorrado repite el resultado de sumar los depósitos y restar los retiros de la meta. Se justifica porque permite mostrar el saldo sin recorrer el historial y porque hace posible aplicar en la base de datos la restricción de saldo no negativo. Para que no se desincronice, solo se modifica dentro de la misma transacción que inserta el movimiento, y un script de verificación comprueba que el saldo sea igual a depósitos menos retiros.

8. Integración de los modelos

8.1 Cadena de coherencia

Las tablas 35 y 36 demuestran la cadena problemática, objetivo central, objetivos específicos, funcionalidades, casos de uso, clases, modelo relacional y arquitectura. Cada fila recorre un mismo hilo: no hay funcionalidad sin casos de uso, casos de uso sin clases, ni clases persistentes sin tabla. El objetivo central (sección 2) se descompone en los cinco objetivos específicos, y estos se distribuyen en las funcionalidades F1 a F6.

Tabla 35

Trazabilidad desde la problemática hasta los casos de uso

Funcionalidad	Necesidad	Objetivo	Casos de uso
F1. Cuenta y acceso	P4	OE1, OE5	CU-01, CU-02, CU-03, CU-04, CU-17, CU-18
F2. Metas de ahorro	P1	OE2	CU-05, CU-06, CU-07
F3. Movimientos financieros	P1, P4	OE2	CU-08, CU-09, CU-10, CU-11, CU-19, CU-20
F4. Educación financiera	P2	OE3	CU-12, CU-13, CU-14, CU-16, CU-21
F5. Modo sin conexión y sincronización	P3	OE4	CU-15 (y el modo sin conexión de CU-05, CU-08, CU-09 y CU-10)
F6. Integridad y seguridad de los datos	P4	OE5	Transversal: restricciones y controles presentes en todos los casos de uso

Nota. Elaboración propia. P1 a P4 son las necesidades de la sección 1.3 y OE1 a OE5 los objetivos específicos de la sección 3.

Tabla 36

Trazabilidad desde las funcionalidades hasta la arquitectura

Funcionalidad	Clases	Tablas	Componentes de la arquitectura
F1	Usuario, Sesion	usuario, sesion	Autenticación y autorización; seguridad transversal; servicios y repositorios
F2	MetaAhorro	meta_ahorro	Controladores, servicios y repositorios; base de datos
F3	Transaccion (Ingreso, Egreso, DepositoMeta, RetiroMeta), CategoriaMovimiento, MetaAhorro	transaccion, categoria_movimiento, meta_ahorro	Servicios (transacciones ACID); repositorios; base de datos
F4	CategoriaLeccion, Leccion, ProgresoLeccion, Logro, LogroObtenido	categoria_leccion, leccion, progreso_leccion, logro, usuario_logro	Módulo de educación (rutas, servicio y repositorio); base de datos
F5	Sincronizacion; codigoGlobal de MetaAhorro y Transaccion	sincronizacion; codigo_global e id_sincronizacion en meta_ahorro y transaccion	SQLite local; cola de sincronización; repositorios; base de datos
F6	Todas las clases	12 claves foráneas, 11 UNIQUE, 23 CHECK y rol de mínimo privilegio	Seguridad transversal; registro estructurado; base de datos

Nota. Elaboración propia. Cada tabla y cada clase pertenece a una única funcionalidad principal.

8.2 Ejemplo integrador: depositar en una meta

Para mostrar cómo colaboran los modelos, la Tabla 37 recorre el caso de uso CU-09 desde la interfaz hasta la base de datos. Cada paso indica el elemento de cada modelo que interviene.

Tabla 37

Recorrido del caso de uso CU-09 Depositar en meta por todos los modelos

Paso	Capa o componente	Elemento del modelo	Qué ocurre
1	Interfaz de usuario	CU-09	El usuario elige «Abonar a meta», selecciona la meta y escribe el monto (por ejemplo, 20.000).
2	Lógica de cliente y SQLite	codigo_global	Valida el monto, genera un UUID y guarda el movimiento en la cola local; si no hay conexión, aquí termina y se sincroniza después.
3	Seguridad y autenticación	Token JWT	La API verifica el token, identifica al usuario y comprueba los límites de uso.
4	Controlador	Esquema de validación	Verifica que la meta y el monto sean válidos y rechaza los campos desconocidos.
5	Servicio de negocio	MetaAhorro.depositar, DepositoMeta	Comprueba que la meta es del usuario y no está pausada ni completada; abre una transacción.
6	Repositorio	transaccion, meta_ahorro	Inserta el movimiento DEPOSITO_META, suma el monto a monto_ahorrado y, si alcanza el objetivo, marca la meta COMPLETADA.
7	Base de datos	CK_transaccion_coherencia, FK_transaccion_meta, CK_meta_ahorro_ahorrado_no_negativo	El motor verifica que el movimiento sea coherente, que la meta pertenezca al usuario y que el saldo no sea negativo.
8	Servicio de negocio	Logro, LogroObtenido (CU-21)	Confirma la transacción y evalúa si corresponde un logro.
9	Interfaz de usuario	MetaAhorro	La API responde con el nuevo saldo y se actualiza la barra de progreso de la meta.

Nota. Elaboración propia. Los pasos 3 a 8 se implementan en la Fase III sobre la estructura de la Fase II.

8.3 Verificación de la correspondencia

La coherencia se verificó de forma automática y no solo por inspección (Tabla 38). El modelo relacional se implementó como script SQL, se ejecutó en SQL Server 2022 y su catálogo se comparó con este documento: los diagramas, el diccionario de datos y las cifras se generaron a partir de ese catálogo.

Tabla 38

Verificaciones realizadas sobre el modelo y su implementación

Verificación	Resultado
Tablas del modelo frente a las clases persistentes	11 tablas y 11 clases persistentes (Transaccion agrupa cuatro especializaciones).
Claves primarias y foráneas	11 claves primarias y 12 foráneas (2 compuestas); coinciden con la sección 7.4.
Restricciones de integridad	11 UNIQUE y 23 CHECK, todas con nombre explícito y documentadas en la sección 7.5.
Datos inválidos (script 98_pruebas_integridad)	29 de 29 operaciones inválidas rechazadas por el motor con la restricción esperada.
Invariante contable (script 99_verificacion)	El saldo de cada meta coincide con depósitos menos retiros; 12 comprobaciones aprobadas.
Datos de prueba frente al prototipo	Ahorro total de $247.500, tres metas activas ($202.000) y los últimos movimientos iguales a los de las pantallas de la Fase I.
Estructura inicial del backend	107 pruebas unitarias y 16 de integración contra SQL Server aprobadas.

Nota. Elaboración propia.

9. Conclusiones

El análisis y el diseño de GKMF permitieron transformar la idea inicial de la Fase I en un conjunto de modelos coherentes entre sí. La problemática de exclusión financiera se descompuso en cuatro necesidades y cada una se tradujo en un objetivo específico, una funcionalidad, casos de uso, clases y tablas que pueden rastrearse de un extremo al otro (sección 8).

La arquitectura en capas con API REST y un cliente *offline-first* responde a la pregunta problema: la integridad y la consistencia se garantizan con transacciones ACID y restricciones en la base de datos; la seguridad, con controles en varias capas alineados con OWASP; y la conectividad limitada, con la réplica local y la sincronización idempotente basada en identificadores globales.

El modelo relacional, de 11 tablas en tercera forma normal, traslada reglas de negocio a la base de datos: las claves foráneas compuestas que aíslan los datos de cada usuario, el libro de movimientos de solo inserción y el saldo de meta no negativo. Su implementación y sus pruebas mostraron que el diseño es viable y que documento y código coinciden.

Una decisión relevante fue sustituir Firebase Firestore por SQL Server como almacén principal, dado que un modelo transaccional con integridad referencial no puede apoyarse en una base documental; Firebase se conserva solo para las notificaciones.

Quedan para las siguientes etapas: implementar las operaciones transaccionales (crear metas, depositar, retirar y sincronizar) con pruebas de concurrencia y aislamiento; completar la autenticación (registro, inicio de sesión, renovación y cierre de sesión) y el bloqueo por intentos fallidos; desarrollar la aplicación móvil y conectar la réplica SQLite; definir la política de tratamiento de datos personales y el consentimiento del usuario; y ampliar la seguridad operativa con copias de respaldo, cifrado en reposo y monitoreo.

10. Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Congreso de la República de Colombia. (2012). *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial 48.587.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gray, J., & Reuter, A. (1993). *Transaction processing: Concepts and techniques*. Morgan Kaufmann.

Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287–317. https://doi.org/10.1145/289.291

Object Management Group. (2017). *Unified Modeling Language (UML) specification* (Version 2.5.1). https://www.omg.org/spec/UML/2.5.1

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

OWASP Foundation. (2023). *OWASP API security top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x00-header/

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Vallejo Bulla, K. J. (2026a). *GKMF – Garantía de Conocimiento y Movilidad Financiera: Documento de contextualización del proyecto* [Documento no publicado]. Universidad Manuela Beltrán.

Vallejo Bulla, K. J. (2026b). *GKMF – Garantía de Conocimiento y Movilidad Financiera* [Código fuente y prototipo]. GitHub. https://github.com/Trabajos-Universidad-Manuela-Beltran/GKMF-Garant-a-de-Conocimiento-y-Movilidad-Financiera

Anexo A. Estructura del repositorio

La Tabla A1 describe la organización del repositorio de GitHub, que evoluciona desde el prototipo de la Fase I hacia la base de datos y el backend inicial (Vallejo Bulla, 2026b).

Tabla A1
Estructura del repositorio del proyecto

Ruta	Contenido
README.md	Descripción del proyecto e instrucciones para ejecutar la base de datos y el backend.
src/	Prototipo de la interfaz (HTML, CSS y JavaScript) de la Fase I.
database/	Scripts T-SQL: 01 crear base de datos, 02 crear tablas, 03 datos iniciales, 04 rol de mínimo privilegio, 90 eliminar esquema, 98 pruebas de integridad y 99 verificación.
backend/	API REST en Node.js, Express y TypeScript: configuración, conexión, middlewares de seguridad, módulos, pruebas y scripts.
docs/	Documentación de la Fase II en PDF y diagramas en PNG.
docker-compose.yml	Entorno de desarrollo: SQL Server, inicialización de la base de datos y API.
.env.example	Plantilla de variables de entorno, sin secretos.

Nota. Elaboración propia.